Written Reflection: CS3081 Cloud Computing Mini Project

Name: Amit Sharma **Roll number:** 230103033
GitHub repository:
https://github.com/amitishusharma2005-prog/cs3081-kd-command-parser

1. What I learned

Sequence-level knowledge distillation trains a small student on the text a larger teacher generates, using ordinary cross-entropy; logit-based distillation would instead match the teacher's token probability distributions, which was impossible here because Phi-3.5 and Pythia-70M use different vocabularies and Ollama exposes only generated text. I learned to shard work with a modulo rule (line number mod SHARD_COUNT) so every command lands in exactly one shard, to keep data cleaning deterministic (sorted ids and a SHA-256 of the output), to separate liveness from readiness so a Pod that is still downloading its model receives no traffic, and that a network ACL is stateless: replies need their own rules and rule numbers decide which match wins.

2. Challenges and how they were handled

The pipeline runner logged these problems while I ran the project: [08:11:56] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:12:29] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:13:04] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:26:55] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:27:26] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:27:58] STAGE_FAILED cluster: command failed (exit 1): bash scripts/secrets.sh | [08:46:20] STAGE_FAILED ollama: cluster pods cannot reach Ollama at host.k3d.internal:11434 -- restart Ollama with OLLAMA_HOST=0.0.0.0 | [08:46:52] STAGE_FAILED ollama: cluster pods cannot reach Ollama at host.k3d.internal:11434 -- restart Ollama with OLLAMA_HOST=0.0.0.0.

3. What I measured

Cleaning kept 2863 of 3000 teacher replies (rejected a=1, b=0, c=10, d=79, duplicates=47). On the 100 validation commands the teacher reached JSON validity 99.0% and full-frame 48.0%, the trained student v1 100.0% and 53.0%, and the untrained Pythia-70M 0.0% and 0.0%. Under the standard Locust run: p50 2000.0 ms, p95 2400.0 ms, 2.687700887365216 requests/s, error rate 0.0%. Cold start median 58 s; rolling update failures 2.151%; pod-recovery failures 0.0%; estimated cost 0.8165 INR per 1,000 requests. Peak memory in the 1 CPU / 1 GiB container test: 990.1 MiB.

4. Observations

The student can only be as good as the labels it is fed: the substring check removes invented slot values but cannot detect a wrong-but-present value, so teacher mistakes pass into training. A single-node k3d cluster shows self-healing, rolling

updates and autoscaling, but it cannot show fault tolerance across machines because every replica shares one laptop. Readiness gating matters most at start-up: the model download dominates cold-start time.

5. What I would improve

Use a larger student or more varied data, add a second-pass check of teacher labels, use dynamic batching or quantisation to cut latency, run a multi-node cluster with a PodDisruptionBudget, replace the static generated S3 keys with short-lived credentials and add TLS on the ingress, and tune requests and limits from the measured CPU and memory usage.